Markdown cell

Assignment 3: Time-Series Weather Forecasting

Brandon Rumbarger

Advanced Machine Learning

Import libraries and set up

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

print("TensorFlow version:", tf.__version__)
print("Pandas version:", pd.__version__)
```

```
from pathlib import Path

DATA_PATH = Path("jena_climate_2009_2016.csv")

data = pd.read_csv(DATA_PATH)

data.head()
```

Load and inspect the dataset

```
print("Dataset shape:", data.shape)
print("\nColumns:")
print(data.columns.tolist())

data.describe()
```

```
# Inspect the temperature column and missing values.
TARGET_COLUMN = "T (degC)"

print(data[TARGET_COLUMN].describe())
print("Missing temperature values:", data[TARGET_COLUMN].isna().sum())
```

Prepare hourly temperature data

```
temperature = data[TARGET_COLUMN].to_numpy(dtype=np.float32)
hourly_temperature = temperature[::6]

n = len(hourly_temperature)
train_end = int(n * 0.50)
val_end = int(n * 0.75)

train_raw = hourly_temperature[:train_end]
val_raw = hourly_temperature[train_end:val_end]
test_raw = hourly_temperature[val_end:]
```

```
train_mean = train_raw.mean()
train_std = train_raw.std()

train_values = (train_raw - train_mean) / train_std
val_values = (val_raw - train_mean) / train_std
test_values = (test_raw - train_mean) / train_std

print("Hourly observations:", n)
print("Train:", len(train_values))
print("Validation:", len(val_values))
print("Test:", len(test_values))
print("Training mean/std:", float(train_mean), float(train_std))
```

```
plt.figure(figsize=(12, 4))
plt.plot(train_raw[:500], label="Training")
plt.plot(np.arange(500, 750), val_raw[:250], label="Validation")
plt.title("Hourly temperature (first 750 plotted observations)")
plt.xlabel("Hourly observation")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.tight_layout()
plt.show()
```

Create forecasting sequences

```
SEQUENCE_LENGTH = 120
FORECAST_DELAY = 24
BATCH_SIZE = 128

def make_sequences(series, sequence_length=SEQUENCE_LENGTH, delay=FORECAST_DELAY):
    X, y = [], []
    # Target is delay hours after the final value in the input window.
    last_start = len(series) - sequence_length - delay + 1
    for start in range(last_start):
        end = start + sequence_length
        target_index = end + delay - 1
        X.append(series[start:end])
        y.append(series[target_index])
    X = np.asarray(X, dtype=np.float32)[..., np.newaxis]
    y = np.asarray(y, dtype=np.float32)
    return X, y

X_train, y_train = make_sequences(train_values)
X_val, y_val = make_sequences(val_values)
X_test, y_test = make_sequences(test_values)

print("Train shapes:", X_train.shape, y_train.shape)
print("Validation shapes:", X_val.shape, y_val.shape)
print("Test shapes:", X_test.shape, y_test.shape)
```

```
def make_dataset(X, y, training=False):
    ds = tf.data.Dataset.from_tensor_slices((X, y))
    if training:
        ds = ds.shuffle(min(len(X), 10000), seed=SEED, reshuffle_each_iteration=True)
    return ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
```

```
train_dataset = make_dataset(X_train, y_train, training=True)
val_dataset = make_dataset(X_val, y_val)
test_dataset = make_dataset(X_test, y_test)
```

Establish a simple baseline

```
baseline_predictions = X_val[:, -1, 0]
baseline_val_mae = np.mean(np.abs(baseline_predictions - y_val))
print(f"Baseline validation MAE: {baseline_val_mae:.4f}")
```

Define model architectures

```
def compile_model(model):
    model.compile(
        optimizer=keras.optimizers.RMSprop(),
        loss="mse",
        metrics=["mae"]
    )
    return model

def build_simple_rnn():
    model = keras.Sequential([
        layers.Input(shape=(SEQUENCE_LENGTH, 1)),
        layers.SimpleRNN(16),
        layers.Dense(1)
    ])
    return compile_model(model)

def build_lstm():
    model = keras.Sequential([
        layers.Input(shape=(SEQUENCE_LENGTH, 1)),
        layers.LSTM(16),
        layers.Dense(1)
    ])
    return compile_model(model)

def build_gru():
    model = keras.Sequential([
        layers.Input(shape=(SEQUENCE_LENGTH, 1)),
        layers.GRU(16),
        layers.Dense(1)
    ])
    return compile_model(model)

def build_stacked_gru(units_1, units_2):
    model = keras.Sequential([
        layers.Input(shape=(SEQUENCE_LENGTH, 1)),
        layers.GRU(units_1, return_sequences=True),
        layers.GRU(units_2),
        layers.Dense(1)
    ])
    return compile_model(model)

def build_cnn_gru():
    model = keras.Sequential([
        layers.Input(shape=(SEQUENCE_LENGTH, 1)),
```

```
        layers.Conv1D(filters=32, kernel_size=5, activation="relu"),
        layers.GRU(16),
        layers.Dense(1)
    ])
    return compile_model(model)
```

Train and compare the models

```
EPOCHS = 10

experiment_builders = {
    "SimpleRNN": build_simple_rnn,
    "LSTM": build_lstm,
    "GRU": build_gru,
    "Stacked GRU (32, 16)": lambda: build_stacked_gru(32, 16),
    "Stacked GRU (64, 32)": lambda: build_stacked_gru(64, 32),
    "Stacked GRU (32, 32)": lambda: build_stacked_gru(32, 32),
    "CNN + GRU": build_cnn_gru,
}

experiment_results = []
trained_models = {}
histories = {}

for model_name, build_fn in experiment_builders.items():
    print(f"\n{'=' * 60}\nTraining: {model_name}\n{'=' * 60}")
    tf.keras.backend.clear_session()
    tf.random.set_seed(SEED)

    model = build_fn()
    history = model.fit(
        train_dataset,
        epochs=EPOCHS,
        validation_data=val_dataset,
        verbose=1
    )

    best_epoch_index = int(np.argmin(history.history["val_mae"]))
    best_val_mae = float(history.history["val_mae"][best_epoch_index])

    experiment_results.append({
        "Model": model_name,
        "Best Epoch": best_epoch_index + 1,
        "Validation MAE": best_val_mae
    })
    trained_models[model_name] = model
    histories[model_name] = history

results_df = pd.DataFrame(experiment_results).sort_values("Validation MAE")
results_df
```

Visualize validation MAE

```
plt.figure(figsize=(10, 5))
plot_results = results_df.sort_values("Validation MAE", ascending=True)
plt.bar(plot_results["Model"], plot_results["Validation MAE"])
```

```
plt.ylabel("Validation MAE (normalized)")
plt.title("Validation MAE by Model")
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.show()
```

Select the best model and evaluate on the test set

```
best_model_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_model_name]
best_validation_mae = float(results_df.iloc[0]["Validation MAE"])

test_loss, test_mae = best_model.evaluate(test_dataset, verbose=0)

print("Best model:", best_model_name)
print(f"Best validation MAE: {best_validation_mae:.4f}")
print(f"Final test MAE: {test_mae:.4f}")
```

Compare selected-model test performance and baseline

```
baseline_test_predictions = X_test[:, -1, 0]
baseline_test_mae = np.mean(np.abs(baseline_test_predictions - y_test))

summary = pd.DataFrame({
    "Method": ["Persistence baseline", best_model_name],
    "Test MAE (normalized)": [baseline_test_mae, test_mae],
    "Approx. Test MAE (°C)": [
        baseline_test_mae * float(train_std),
        test_mae * float(train_std)
    ]
})
summary
```

Plot selected model predictions

```
predictions = best_model.predict(test_dataset, verbose=0).reshape(-1)

# Show a small segment to make the forecast and actual series readable.
n_plot = min(300, len(y_test))
actual_c = y_test[:n_plot] * train_std + train_mean
predicted_c = predictions[:n_plot] * train_std + train_mean

plt.figure(figsize=(12, 5))
plt.plot(actual_c, label="Actual temperature")
plt.plot(predicted_c, label="Predicted temperature")
plt.title(f"Test-set temperature predictions: {best_model_name}")
plt.xlabel("Test sequence")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.tight_layout()
plt.show()
```